МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-67

Выполнил
В. В. Давидюк,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Вариант 3

Вариант 3 Классификация качества вина

- **Набор данных:** Wine Quality
- **Задача:** классифицировать вино на "хорошее" (оценка >= 7) и "обычное" (бинарная классификация).
- **Архитектура:**
  - входной слой;
  - два скрытых слоя по 12 нейронов в каждом (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** увеличьте количество эпох обучения в два раза и посмотрите, привело ли это к улучшению F1-score.

Выполнение:

Код программы

```
import copy
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

torch.manual_seed(42)
np.random.seed(42)

data = pd.read_csv("winequality-white.csv", sep=";")

X = data.drop(columns=["quality"]).values
y = (data["quality"] >= 7).astype(int).values

print("Количество записей:", len(data))
print("Количество признаков:", X.shape[1])
print("Обычное вино:", np.sum(y == 0))
print("Хорошее вино:", np.sum(y == 1))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train.reshape(-1, 1), dtype=torch.float32)
y_test = torch.tensor(y_test.reshape(-1, 1), dtype=torch.float32)

class MLP(nn.Module):
    def __init__(self, input_size):
        super().__init__()
```

```
        self.layers = nn.Sequential(
            nn.Linear(input_size, 12),
            nn.ReLU(),
            nn.Linear(12, 12),
            nn.ReLU(),
            nn.Linear(12, 1)
        )

    def forward(self, x):
        return self.layers(x)

model = MLP(X_train.shape[1])

pos_weight = torch.tensor(
    [float((y_train == 0).sum() / (y_train == 1).sum())],
    dtype=torch.float32
)

criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
optimizer = optim.Adam(model.parameters(), lr=0.001)

state_100 = None

for epoch in range(1, 201):
    model.train()

    y_pred = model(X_train)
    loss = criterion(y_pred, y_train)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch == 100:
        state_100 = copy.deepcopy(model.state_dict())

    if epoch % 50 == 0:
        print(f"Эпоха {epoch}, Loss: {loss.item():.4f}")

state_200 = copy.deepcopy(model.state_dict())

def evaluate(state, epochs):
    test_model = MLP(X_train.shape[1])
    test_model.load_state_dict(state)
    test_model.eval()

    with torch.no_grad():
        output = test_model(X_test)
        probabilities = torch.sigmoid(output)
        predictions = (probabilities >= 0.5).int().numpy().flatten()

    actual = y_test.numpy().flatten()

    accuracy = accuracy_score(actual, predictions)
    precision = precision_score(actual, predictions, zero_division=0)
    recall = recall_score(actual, predictions, zero_division=0)
    f1 = f1_score(actual, predictions, zero_division=0)
    matrix = confusion_matrix(actual, predictions)

    print(f"\nРЕЗУЛЬТАТЫ ПОСЛЕ {epochs} ЭПОХ")
    print(f"Accuracy:  {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-score:  {f1:.4f}")
    print("Confusion Matrix:")
    print(matrix)

    return f1
```

```
f1_100 = evaluate(state_100, 100)
f1_200 = evaluate(state_200, 200)

print("\nСРАВНЕНИЕ РЕЗУЛЬТАТОВ")
print(f"F1-score (100 эпох): {f1_100:.4f}")
print(f"F1-score (200 эпох): {f1_200:.4f}")

if f1_200 > f1_100:
    print("Увеличение количества эпох улучшило F1-score.")
elif f1_200 < f1_100:
    print("Увеличение количества эпох ухудшило F1-score.")
else:
    print("Увеличение количества эпох не изменило F1-score.")
```

Результат:

Количество записей: 4898
Количество признаков: 11
Обычное вино: 3838
Хорошее вино: 1060
Эпоха 50, Loss: 1.0475
Эпоха 100, Loss: 0.9615
Эпоха 150, Loss: 0.8562
Эпоха 200, Loss: 0.7941

РЕЗУЛЬТАТЫ ПОСЛЕ 100 ЭПОХ
Accuracy: 0.6561
Precision: 0.3550
Recall: 0.7217
F1-score: 0.4759
Confusion Matrix:
[[490 278]
[59 153]]

РЕЗУЛЬТАТЫ ПОСЛЕ 200 ЭПОХ
Accuracy: 0.7418
Precision: 0.4388
Recall: 0.6934
F1-score: 0.5375
Confusion Matrix:
[[580 188]
[65 147]]

СРАВНЕНИЕ РЕЗУЛЬТАТОВ
F1-score (100 эпох): 0.4759
F1-score (200 эпох): 0.5375
Увеличение количества эпох улучшило F1-score.

Вывод: В результате эксперимента было установлено, что увеличение количества эпох обучения со 100 до 200 привело к повышению значения F1-score с 0,4759 до 0,5375, а Accuracy увеличилась с 0,6561 до 0,7418. Несмотря на небольшое снижение Recall, общее качество классификации улучшилось. Таким образом, цель лабораторной работы достигнута, а увеличение количества эпох положительно повлияло на результаты работы модели.